

Rapport Labo 1

Kenan Augsburgur

22 September 2025

Contents

1. Installation and Launch	3
2. Main App Description	5
3. Creating Groups	6

1. Installation and Launch

? What are tokens used for and how do they look like in OpenBao?

Tokens are used to authenticate when accessing the vault. Tokens can be used directly or generated dynamically through auth methods. (Using configured authentication providers like github OAuth)

The example provided in the documentation for token formats is the following

```
1 b.n6keuKu5Q6pXhaIcfnC9cFNd
2 r.JaKnR2AIHNk3fC4SGyyyDVoQ90
3 s.raPGTZdARXdY0KvHcWSpp5wWZIHNT
```

Plain Text

Where each token is prefixed according to their type.

- s. : Service tokens
- b. : Batch tokens
- r. : Recovery tokens

A token's body is a random Base62 value of 24 characters.

[Tokens documentation](#)

i Note

Root tokens are tokens with the root policy attached. Those tokens provide access to anything within OpenBao and can be set to never expire without any renewal required.

Those tokens are hard to create and should only be used for the initial setup or emergencies. They should be revoked otherwise.

1. The initial root token is generated at `bao operator init` time and has no expiration
2. Root tokens can create other root tokens. Tokens with an expiration cannot create a token without expiration.
3. Using `bao operator generate-root` with permission of a quorum of unseal key holders. (Similar to the key ceremony I guess)

i Environment setup

```
1 docker compose up -d
2 set -x BAO_ADDR http://localhost:8200
3 # Login prompt, use a token when prompted
4 bao login
5 bao status
```

Fish

? How do you set up what are the capabilities of a token?

Capabilities are a property of policies. With policies you can define what can be done based on a path. Once you have a policy you can apply the policy to a token.

For example, you can create the policy `foo`.

```
1 # file: foo.hcl
2 # Allow read of anything within secret/foo
3 path "secret/foo/*" {
4     capabilities = ["read"]
5 }
6
7 # Allow read and update for secret/bar
8 path "secret/bar" {
9     capabilities = ["read", "update"]
10 }
```

 Terraform

```
1 bao policy write foo foo.hcl
```

 Fish

And create a token with the new policy

```
1 bao token create -display-name foo -policy foo
2 # Key Value
3 # ---
4 # token s.GHmDKZEh0TKGJQ6I5McAvoy9
5 # token_accessor v0EJuFUt81Hy6J7q3YE74xJh
6 # token_duration 768h
7 # token_renewable true
8 # token_policies ["default" "foo"]
9 # identity_policies []
10 # policies ["default" "foo"]
```

 Fish

[Policy syntax](#)^o

? What is sealing and unsealing and what is it used for?

Sealing means that the server will throw away the root key it was using to decipher the secrets. This means that an intruder cannot extract the key from a memory dump but it also means that the service won't be available anymore.

Unsealing is the process of deciphering the root key. This requires the use of unsealing keys which are shared between multiple people using Shamir's secret sharing. (Similar to the key ceremony for the root CAs.)

As stated, the goal of sealing is to ensure that secrets won't leak if signs of intrusion are detected.

2. Main App Description



Why isn't it a great idea to store the ciphertexts on Vault? (We do it to avoid having to setup a database).

The vault's goal is to be a place to securely store secrets. As such it's optimized for security before performances. Accessing each message and metadata in the vault is done through the http(s) api and it doesn't seem like you can read in batch. This means that one message results in one http call (which will in turn slow down the vault as the number users/messages grows).

And more importantly, each message is stored as a ciphertext which means it's already encrypted. Storing it in the vault means that there are two layers of encryption which increases processing time without any meaningful security gains.



Quick setup

I created a `fish` script to quickly setup the dev server with the kv and transit engines

```
1 ./setup.fish
```



3. Creating Groups

? What is key rotation and what is it used for?

A key rotation consists of changing the key used for encryption. Once rotated, the old key isn't used to encrypt anymore but it can still be used to decrypt ciphertexts that were encrypted with it.

The rotation is helpful against cryptanalysis as well as potential breaches. The likelihood of cryptanalysis to work scales with the number of known ciphertexts. By rotating the key we limit the number of ciphertexts which in turn reduces the probability of cryptanalysis to work.

If the key is compromised, at least the affected data is limited to the period between key rotations and not the whole database.

? How does Vault make sure one can decrypt an old message when keys are rotated?

The key is kept. Each ciphertext has metadata attached to it which contains all the informations necessary to determine which version (rotation) of the key should be used when deciphering

? What do you think of having a key rotation every 1h?

This seems aggressive. The documentation mentions the NIST rotation guidance which tells us that the rotation should happen before a number of encryptions have been made with a key. This number of encryptions depends on the algorithm of the key but in their example a rotation every 3 month with 40M operations per day is sufficient.

Our application won't come anywhere close to that so having 1h rotation is only good to prove that the rotation works but for a production application it will just generate a lot of keys which could potentially reduce performances.

```
1 # Financial.hcl
2 # Allow encryption using the Financial key
3 path "transit/encrypt/Financial" {
4     capabilities = ["update"]
5 }
6
7 # Allow decryption using the Financial key
8 path "transit/decrypt/Financial" {
9     capabilities = ["update"]
10 }
11
12 # Allow read and write access to the Financial store
13 path "kv-v2/data/ciphertexts/Financial/*" {
```

 Terraform

```
14     capabilities = ["read", "create"]
15 }
```

```
1  # IT.hcl
2  # Allow encryption using the IT key
3  path "transit/encrypt/IT" {
4      capabilities = ["update"]
5  }
6
7  # Allow decryption using the IT key
8  path "transit/decrypt/IT" {
9      capabilities = ["update"]
10 }
11
12 # Allow read and write access to the IT store
13 path "kv-v2/data/ciphertexts/IT/*" {
14     capabilities = ["read", "create"]
15 }
```

 Terraform



What capabilities did you give for the transit service and why? Provide the policy file in your report

`update`. The policies documentation states `POST` and `PUT` methods are used for `create` and `update` as well as the fact that most of the project doesn't distinguish between the two and require both.

When looking at the transit API for encrypt, it's one of the functionalities which distinguish between the two. With `update` only, the encryption will fail if the key wasn't created. On the other hand, with `create`, the key will be generated with default values (for parameters.) Since we want specific types for the keys, it's better to only use `update`.



What capabilities did you give for the kv service and why? Provide the policy file in your report.

`create`, `read` and `list`. The instructions stated that users should be able to read and write to the kv (under their respective groups.) Whether they should be allowed to `update` or `delete` is ambiguous and will be restricted until clarification. The `list` capability is added since the kv will be used as the database and the application won't know the keys otherwise.